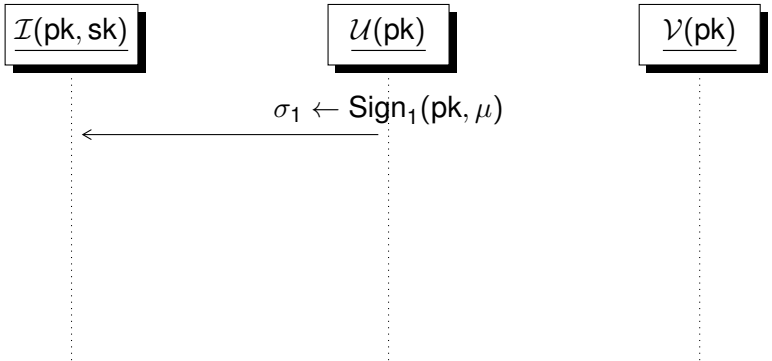

DTU

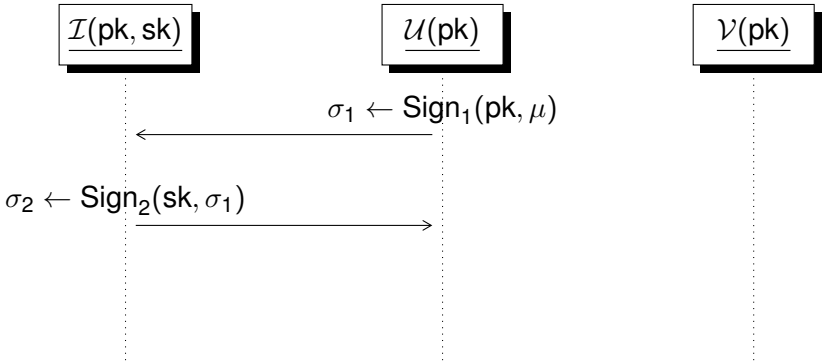
Carsten Baum, **Marvin Beckmann**

edaBits in the Exponent (edaBitE) and Issuer-Hiding Blind Signatures from ML-DSA and VOLE-in-the-Head

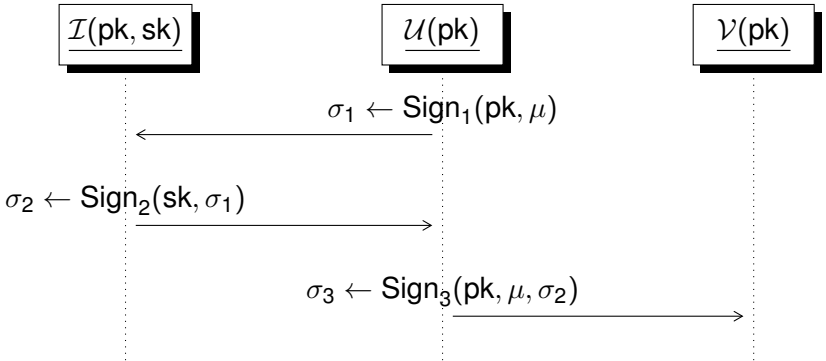
Blind Signatures



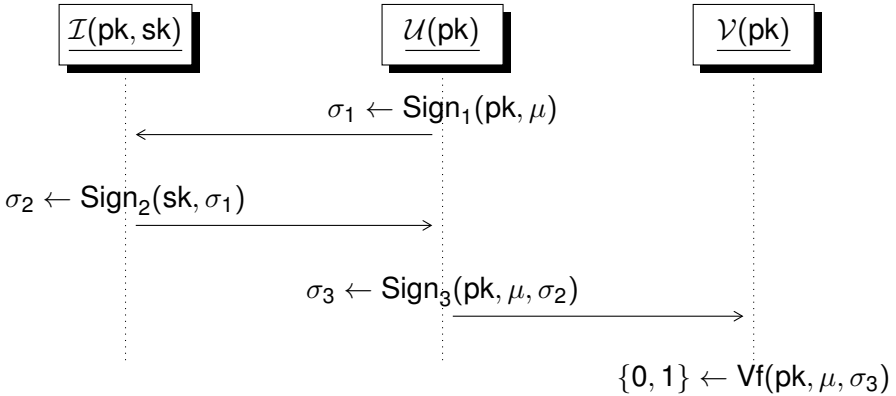
Blind Signatures



Blind Signatures

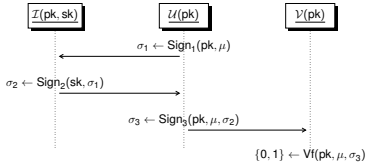


Blind Signatures



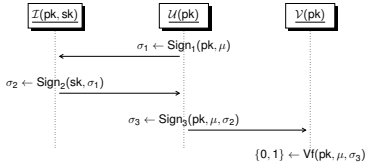
Fischlin's Blueprint for Blind Signatures

- (pk, sk) are from a normal signature scheme



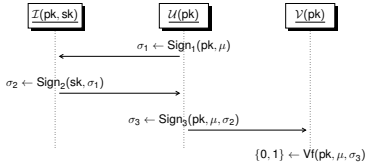
Fischlin's Blueprint for Blind Signatures

- $(\mathsf{pk}, \mathsf{sk})$ are from a normal signature scheme
- $\sigma_1 \leftarrow \mathit{Com}(\mu, r)$ where $r \leftarrow \{0,1\}^{1^\lambda}$



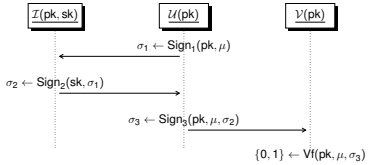
Fischlin's Blueprint for Blind Signatures

- $(\mathsf{pk}, \mathsf{sk})$ are from a normal signature scheme
- $\sigma_1 \leftarrow \mathit{Com}(\mu, r)$ where $r \leftarrow \{0,1\}^{1^\lambda}$
- $\sigma_2 \leftarrow \mathsf{Sign}(\mathsf{sk}, \sigma_1)$



Fischlin's Blueprint for Blind Signatures

- $(\mathsf{pk}, \mathsf{sk})$ are from a normal signature scheme
- $\sigma_1 \leftarrow \mathit{Com}(\mu, r)$ where $r \leftarrow \{0,1\}^{1^\lambda}$
- $\sigma_2 \leftarrow \mathsf{Sign}(\mathsf{sk}, \sigma_1)$
- $\sigma_3 \leftarrow (\mu, z)$ where $z \leftarrow \mathit{ZKP}(\mathsf{pk}, \sigma_2, \mu, r)$



Why use Fischlin?

- Fischlin's blueprint *can use* NIST-standards: ML-DSA [Nat24]

Why use Fischlin?

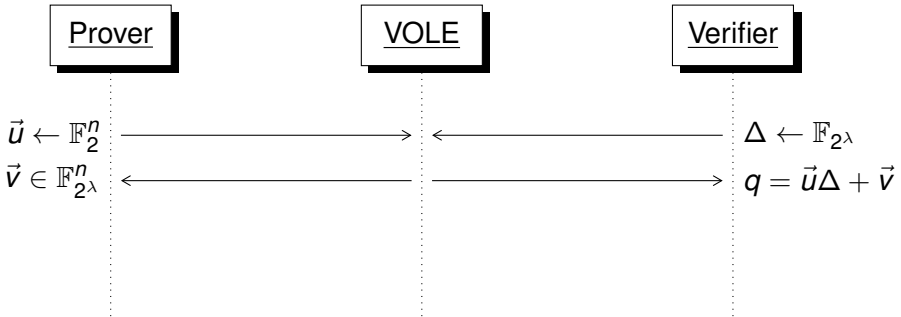
- Fischlin's blueprint *can use* NIST-standards: ML-DSA [Nat24]
- Issuer runs an *unmodified* ML-DSA implementation

Why use Fischlin?

- Fischlin's blueprint *can use* NIST-standards: ML-DSA [Nat24]
- Issuer runs an *unmodified* ML-DSA implementation
- Difficulty: the circuit must prove *exactly* ML-DSA.Verify

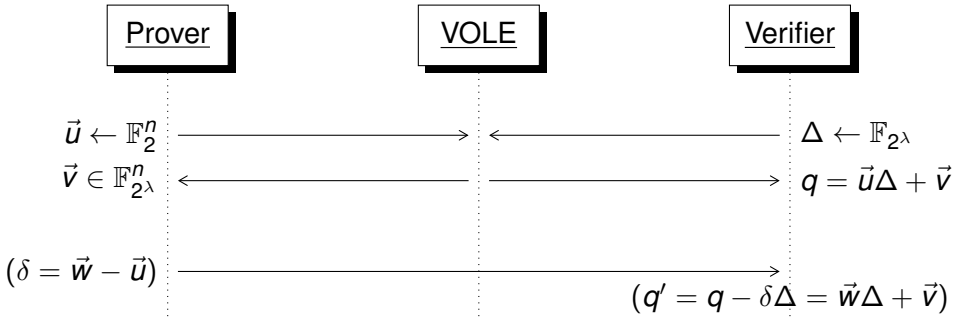
Vector Oblivious Linear Evaluation (VOLE)

Vector Oblivious Linear Evaluation (VOLE)



▷ The verifier evaluates the polynomial $p(x) = \vec{u}x + \vec{v}$ at position Δ.

Vector Oblivious Linear Evaluation (VOLE)



▷ The verifier evaluates the polynomial $p(x) = \vec{u}x + \vec{v}$ at position Δ.

Properties of VOLEitH as a Commit&Proof Framework

Let d_a be the degree of $[a]$ and d_b be the degree of $[b]$, where $d_a \geq d_b$

Properties of VOLEitH as a Commit&Proof Framework

Let d_a be the degree of $[a]$ and d_b be the degree of $[b]$, where $d_a \geq d_b$

Linear

$$[a] + [b]\Delta^{d_a - d_b} = [a + b]$$

Properties of VOLEitH as a Commit&Proof Framework

Let d_a be the degree of $[a]$ and d_b be the degree of $[b]$, where $d_a \geq d_b$

Linear

$$[a] + [b]\Delta^{d_a - d_b} = [a + b]$$

Multiplication Is a degree-$(d_a + d_b)$ commitment to ab

$$[a] \cdot [b]$$

Properties of VOLEitH as a Commit&Proof Framework

Let d_a be the degree of $[a]$ and d_b be the degree of $[b]$, where $d_a \geq d_b$

Linear

$$[a] + [b]\Delta^{d_a - d_b} = [a + b]$$

Multiplication Is a degree-$(d_a + d_b)$ commitment to ab

$$[a] \cdot [b]$$

Assert $a = 0$ reveal lower-degree coefficient masked by $d - 1$ fresh $\mathbb{F}_{\text{ext}}$-VOLEs

Properties of VOLEitH as a Commit&Proof Framework

Let d_a be the degree of $[a]$ and d_b be the degree of $[b]$, where $d_a \geq d_b$

Linear

$$[a] + [b]\Delta^{d_a - d_b} = [a + b]$$

Multiplication Is a degree-$(d_a + d_b)$ commitment to ab

$$[a] \cdot [b]$$

Assert $a = 0$ reveal lower-degree coefficient masked by $d - 1$ fresh $\mathbb{F}_{\text{ext}}$-VOLEs

Degree reduct. re-commit a fresh at degree 1, $[a]'$, and assert $[a] - [a]'\Delta^{d_a - 1} = 0$

Prior Work: Blind Signatures from MAYO

Prior Work: Blind Signatures from MAYO

- PoMFRIT [BBB+26]: Fischlin blueprint, instantiated with MAYO
- MAYO verification is a degree-2 check on the signature and one evaluation of SHAKE256

Prior Work: Blind Signatures from MAYO

- PoMFRIT [BBB+26]: Fischlin blueprint, instantiated with MAYO
- MAYO verification is a degree-2 check on the signature and one evaluation of SHAKE256

▷ what does it take to do the same for ML-DSA?

ML-DSA **Verification**

Algorithm ML-DSA.Verify(pk, M′, σ)

Require: Public key $(\hat{\boldsymbol{A}}, \boldsymbol{tr}, \boldsymbol{t}_1)$, message M′, signature $(\tilde{\boldsymbol{c}}, \boldsymbol{z}, \boldsymbol{h})$

1: $\boldsymbol{\mu} \leftarrow \text{SHAKE256}(\boldsymbol{tr} \parallel \text{M}')$
2: $\boldsymbol{c} \leftarrow \text{SampleInBall}(\tilde{\boldsymbol{c}})$
3: $\boldsymbol{w}_{\text{appr}} \leftarrow \hat{\boldsymbol{A}} \cdot \boldsymbol{z} - \boldsymbol{c} \cdot \boldsymbol{t}_1 \cdot 2^{13}$
4: $\boldsymbol{w}_1 \leftarrow \text{UseHint}(\boldsymbol{h}, \boldsymbol{w}_{\text{appr}})$
5: $\tilde{\boldsymbol{c}}' \leftarrow \text{SHAKE256}(\boldsymbol{\mu} \parallel \text{w1Encode}(\boldsymbol{w}_1))$
6: **return** 1 iff $\|\boldsymbol{z}\|_\infty \leq \gamma_z$ and $\tilde{\boldsymbol{c}} = \tilde{\boldsymbol{c}}'$

- Lines 1, 5: SHAKE256 ✓

ML-DSA **Verification**

Algorithm ML-DSA.Verify($\mathsf{pk}, \mathrm{M}', \sigma$)

Require: Public key $(\hat{\boldsymbol{A}}, \boldsymbol{tr}, \boldsymbol{t}_1)$, message M', signature $(\tilde{\boldsymbol{c}}, \boldsymbol{z}, \boldsymbol{h})$

1: $\boldsymbol{\mu} \leftarrow \mathrm{SHAKE256}(\boldsymbol{tr} \parallel \mathrm{M}')$
2: $\boldsymbol{c} \leftarrow \mathrm{SampleInBall}(\tilde{\boldsymbol{c}})$
3: $\boldsymbol{w}_{\mathrm{appr}} \leftarrow \hat{\boldsymbol{A}} \cdot \boldsymbol{z} - \boldsymbol{c} \cdot \boldsymbol{t}_1 \cdot 2^{13}$
4: $\boldsymbol{w}_1 \leftarrow \mathrm{UseHint}(\boldsymbol{h}, \boldsymbol{w}_{\mathrm{appr}})$
5: $\tilde{\boldsymbol{c}}' \leftarrow \mathrm{SHAKE256}(\boldsymbol{\mu} \parallel \mathrm{w1Encode}(\boldsymbol{w}_1))$
6: **return** 1 iff $\|\boldsymbol{z}\|_\infty \leq \gamma_z$ and $\tilde{\boldsymbol{c}} = \tilde{\boldsymbol{c}}'$

- Lines 1, 5: SHAKE256 ✓
- Line 2: SampleInBall ?
- Line 4: UseHint – conversion between $\mathbb{F}_2$ and $\mathbb{F}_q$
- Line 6: a norm bound on $\boldsymbol{z}$

ML-DSA **Verification**

Algorithm ML-DSA.Verify(pk, M′, σ)

Require: Public key $(\hat{\boldsymbol{A}}, \boldsymbol{tr}, \boldsymbol{t}_1)$, message M′, signature $(\tilde{\boldsymbol{c}}, \boldsymbol{z}, \boldsymbol{h})$

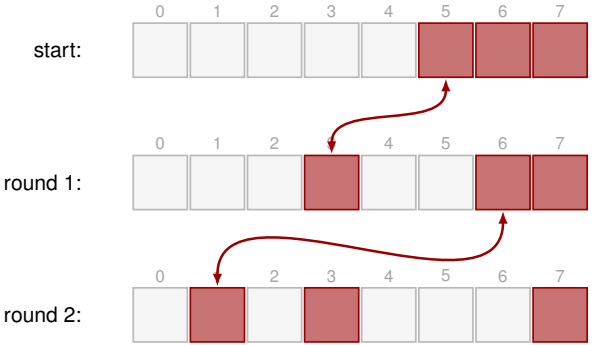
1: $\boldsymbol{\mu} \leftarrow \text{SHAKE256}(\boldsymbol{tr} \parallel \text{M}')$
2: $\boldsymbol{c} \leftarrow \text{SampleInBall}(\tilde{\boldsymbol{c}})$
3: $\boldsymbol{w}_{\text{appr}} \leftarrow \hat{\boldsymbol{A}} \cdot \boldsymbol{z} - \boldsymbol{c} \cdot \boldsymbol{t}_1 \cdot 2^{13}$
4: $\boldsymbol{w}_1 \leftarrow \text{UseHint}(\boldsymbol{h}, \boldsymbol{w}_{\text{appr}})$
5: $\tilde{\boldsymbol{c}}' \leftarrow \text{SHAKE256}(\boldsymbol{\mu} \parallel \text{w1Encode}(\boldsymbol{w}_1))$
6: **return** 1 iff $\|\boldsymbol{z}\|_\infty \leq \gamma_z$ and $\tilde{\boldsymbol{c}} = \tilde{\boldsymbol{c}}'$

- Lines 1, 5: SHAKE256 ✓
- Line 2: SampleInBall ?
- Line 4: UseHint – conversion between $\mathbb{F}_2$ and $\mathbb{F}_q$
- Line 6: a norm bound on $\boldsymbol{z}$

▷ The rest has no extra cost

Proving SampleInBall

SampleInBall



- Squeeze a stream from SHAKE256($\tilde{\boldsymbol{c}}$): 8 sign bytes per attempt
- For $i = N - \nu, \ldots, N - 1$: draw j from the stream until $j \leq i$, then swap

General ZK-ORAM [YH24]

- Every array access (read or write) is logged $(\text{address}, \text{value}, \text{timestamp})$
- Reads are retrieved from a list reads, writes append to a list writes

General ZK-ORAM [YH24]

- Every array access (read or write) is logged $(\text{address}, \text{value}, \text{timestamp})$
- Reads are retrieved from a list reads, writes append to a list writes
- The consistency of the lists is checked with a random challenge.

General ZK-ORAM [YH24]

- Every array access (read or write) is logged $(\text{address}, \text{value}, \text{timestamp})$
- Reads are retrieved from a list reads, writes append to a list writes
- The consistency of the lists is checked with a random challenge.

▷ Each shuffle round: 2 reads and 2 writes per round, N writes for initialization and N final reads.

General ZK-ORAM [YH24]

- Every array access (read or write) is logged (address, value, timestamp)
- Reads are retrieved from a list reads, writes append to a list writes
- The consistency of the lists is checked with a random challenge.

▷ Each shuffle round: 2 reads and 2 writes per round, N writes for initialization and N final reads.

ORAM	$\theta_{\max}$	Prove Time (ms)	Verify Time (ms)	Size (KB)
General	128	81.06	56.28	29.347
	213	85.97	61.74	36.367

From $\mathbb{F}_2$ to $\mathbb{F}_q$: Conversions

Bit Commitment as Exponent

Lemma

For bits $b_1, \ldots, b_k \in \{0,1\}$, $b = \sum_{j\in[k]} b_j 2^{j-1}$ *and any* $X \in \mathbb{F}_{2^\lambda}$,

$$\prod_{j\in[k]} \mathsf{elift}_X(2^{j-1}, b_j) = X^b, \qquad \mathsf{elift}_X(exp, a) := 1 + (X^{exp} + 1) \cdot a = X^{exp \cdot a}.$$

Bit Commitment as Exponent

Lemma

For bits $b_1, \ldots, b_k \in \{0,1\}$, $b = \sum_{j\in[k]} b_j 2^{j-1}$ *and any* $X \in \mathbb{F}_{2^\lambda}$,

$$\prod_{j\in[k]} \mathsf{elift}_X(2^{j-1}, b_j) = X^b, \qquad \mathsf{elift}_X(exp, a) := 1 + (X^{exp} + 1) \cdot a = X^{exp \cdot a}.$$

- Lifts committed $\mathbb{F}_2$ bits into a value *in the exponent* of $\mathbb{F}_{2^\lambda}$

Bit Commitment as Exponent

Lemma

For bits $b_1, \ldots, b_k \in \{0,1\}$, $b = \sum_{j\in[k]} b_j 2^{j-1}$ *and any* $X \in \mathbb{F}_{2^\lambda}$,

$$\prod_{j\in[k]} \mathsf{elift}_X(2^{j-1}, b_j) = X^b, \qquad \mathsf{elift}_X(exp, a) := 1 + (X^{exp} + 1) \cdot a = X^{exp \cdot a}.$$

- Lifts committed $\mathbb{F}_2$ bits into a value *in the exponent* of $\mathbb{F}_{2^\lambda}$
- High-level idea: do a random linear check in $\mathbb{F}_q$ and the exponent of $\mathbb{F}_2$.

The *edaBitE* Check

Draw $\chi_1, \ldots, \chi_n \leftarrow \mathbb{F}_q$.

$$e := \sum_{i \in [n]} \chi_i x_i \bmod q, \qquad P := \prod_{i \in [n]} \prod_{j \in [k_i]} \mathsf{elift}_{X^{\chi_i}}\left(2^{j-1}, [x_{i,j}]_2\right) = X^{\sum_{i \in [n]} \chi_i x_i}$$

The *edaBitE* Check

Draw $\chi_1, \ldots, \chi_n \leftarrow \mathbb{F}_q$.

$$e := \sum_{i \in [n]} \chi_i x_i \bmod q, \qquad P := \prod_{i \in [n]} \prod_{j \in [k_i]} \mathsf{elift}_{X^{\chi_i}}\left(2^{j-1}, [x_{i,j}]_2\right) = X^{\sum_{i \in [n]} \chi_i x_i}$$

- The prover reveals the integer $E := \sum_{i \in [n]} \chi_i x_i$ for which $P = X^E$

The *edaBitE* Check

Draw $\chi_1, \ldots, \chi_n \leftarrow \mathbb{F}_q$.

$$e := \sum_{i \in [n]} \chi_i x_i \bmod q, \qquad P := \prod_{i \in [n]} \prod_{j \in [k_i]} \mathsf{elift}_{X^{\chi_i}}\left(2^{j-1}, [x_{i,j}]_2\right) = X^{\sum_{i \in [n]} \chi_i x_i}$$

- The prover reveals the integer $E := \sum_{i \in [n]} \chi_i x_i$ for which $P = X^E$
- The verifier accepts iff $0 \leq E < \mathrm{ord}(X)$, $P = X^E$, and $E \equiv e \pmod q$

edaBitE vs. MYSTIQUE: Cost Comparison

- MYSTIQUE[WYX+21] (cut-and-choose): commit the pair, then *sacrifice* and open; inefficient non-interactively
- edaBitE: commit the pair once, certify a whole batch together via the exponent trick above; only degree reductions over $\mathbb{F}_{2^\lambda}$ are needed

	MYSTIQUE	edaBitE
512 candidate pairs, full width	47.1 KB	47.1 KB
Certification overhead	632.7 KB	21.1 KB
Total	679.8 KB	68.2 KB

The Range Check on z

- Need: $\|\boldsymbol{z}\|_\infty \leq \gamma_z$, i.e. each shifted coefficient $z_j + \gamma_z \in [0, 2\gamma_z]$
- Classic approach: commit digit representation in $\mathbb{F}_q$ and compare

The Range Check on z

- Need: $\|\boldsymbol{z}\|_\infty \leq \gamma_z$, i.e. each shifted coefficient $z_j + \gamma_z \in [0, 2\gamma_z]$
- Classic approach: commit digit representation in $\mathbb{F}_q$ and compare
- With edaBitE, conversions are cheap: commit the bits in $\mathbb{F}_2$ instead, and recover $[\boldsymbol{z}]_q$ with *one* conversion per coordinate

The Range Check on z

- Need: $\|\boldsymbol{z}\|_\infty \leq \gamma_z$, i.e. each shifted coefficient $z_j + \gamma_z \in [0, 2\gamma_z]$
- Classic approach: commit digit representation in $\mathbb{F}_q$ and compare
- With edaBitE, conversions are cheap: commit the bits in $\mathbb{F}_2$ instead, and recover $[\boldsymbol{z}]_q$ with *one* conversion per coordinate

Variant	(B, ℓ)	Prove Time (ms)	Verify Time (ms)	Size (KB)
BitWise	$(2, 18)$	61.00	50.55	858.657
TwoBits	$(4, 9)$	51.70	30.07	437.205
ThreeBits	$(8, 6)$	78.15	25.54	300.033
FourBits	$(16, 5)$	198.35	27.02	262.037
FiveBits	$(32, 4)$	481.93	31.91	228.457
SixBits	$(64, 3)$	1865.60	39.70	199.293
edaBitE	$(2, 18)$	1416.20	754.99	133.472
Mystique	$(2, 18)$	2508.70	287.23	1268.237

Issuer-Hiding & Attribute Disclosure

Issuer-Hiding & Attribute Disclosure

Issuer-hiding replace the single public key by a public *table* of n issuers; the prover selects one row via a secret, bit-committed index

Issuer-Hiding & Attribute Disclosure

Issuer-hiding replace the single public key by a public *table* of n issuers; the prover selects one row via a secret, bit-committed index

Attributes the issuer additionally signs hidden attributes; the prover proves a public predicate over them without opening the attributes themselves

Issuer-Hiding & Attribute Disclosure

Issuer-hiding replace the single public key by a public *table* of n issuers; the prover selects one row via a secret, bit-committed index

Attributes the issuer additionally signs hidden attributes; the prover proves a public predicate over them without opening the attributes themselves

total issuers n	Runtime in ms			Proof Size (KB)
	Sig_1	Sig_3	*Ver*	
baseline	197.41	815.29	504.04	1011.909
27	202.28/209.99	873.74/874.76	522.96/519.41	1059.253/1086.629
128	198.93/202.56	942.56/964.68	561.52/569.10	1059.350/1086.726
512	201.72/204.74	1306.00/1311.30	760.25/766.39	1059.445/1086.821

Issuer-Hiding & Attribute Disclosure

Issuer-hiding replace the single public key by a public *table* of n issuers; the prover selects one row via a secret, bit-committed index

Attributes the issuer additionally signs hidden attributes; the prover proves a public predicate over them without opening the attributes themselves

total issuers n	Runtime in ms			Proof Size (KB)
	Sig_1	Sig_3	*Ver*	
baseline	197.41	815.29	504.04	1011.909
27	202.28/209.99	873.74/874.76	522.96/519.41	1059.253/1086.629
128	198.93/202.56	942.56/964.68	561.52/569.10	1059.350/1086.726
512	201.72/204.74	1306.00/1311.30	760.25/766.39	1059.445/1086.821

Thanks! eprint: 2026/2473

References I

- Carsten Baum, Marvin Beckmann, Ward Beullens, Shibam Mukherjee, and Christian Rechberger.
  Concretely efficient blind signatures based on {VOLE-in-the-Head} proofs and the {MAYO} trapdoor.
  In *35th USENIX Security Symposium (USENIX Security 26)*, pages 6325–6344, 2026.
- National Institute of Standards and Technology.
  Module-lattice-based digital signature standard.
  Technical Report NIST Federal Information Processing Standards (FIPS) 204, U.S. Department of Commerce, Gaithersburg, MD, aug 2024.

References II

- Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang.
  Mystique: Efficient conversions for zero-knowledge proofs with applications to machine learning.
  pages 501–518, 2021.
- Yibin Yang and David Heath.
  Two shuffles make a RAM: Improved constant overhead zero knowledge RAM.
  2024.